WEB APPLICATION HACKING AND DEFENSE: A STUDY GUIDE

Understand the request. Recognize the weakness. Choose the control.



Study the application and the infrastructure together: either can expose the other.

How to use this guide

Start with architecture, then connect each attack to its underlying trust failure. Review the testing workflow and tool families before matching weaknesses to preventive and detective controls.

For exam scenarios, identify the affected layer, the attacker's capability, and the most direct mitigation. Distinguish a root-cause fix from a compensating control.

THE STUDY ROUTE

- Systems and attack surfaces
- Threats and testing methods
- Tools and countermeasures
- Verification and retesting

> Practice only in isolated labs or on systems covered by explicit authorization. Agree scope, permitted techniques, and stop conditions before testing.

Web apps are layered systems, and every layer can be attacked

A web application moves untrusted requests through several components. Each boundary needs its own security checks.

The three tiers

- The client browser renders pages and runs JavaScript.
- The web server and application logic route requests and enforce business rules.
- The database stores and retrieves persistent data.

Why attackers target them

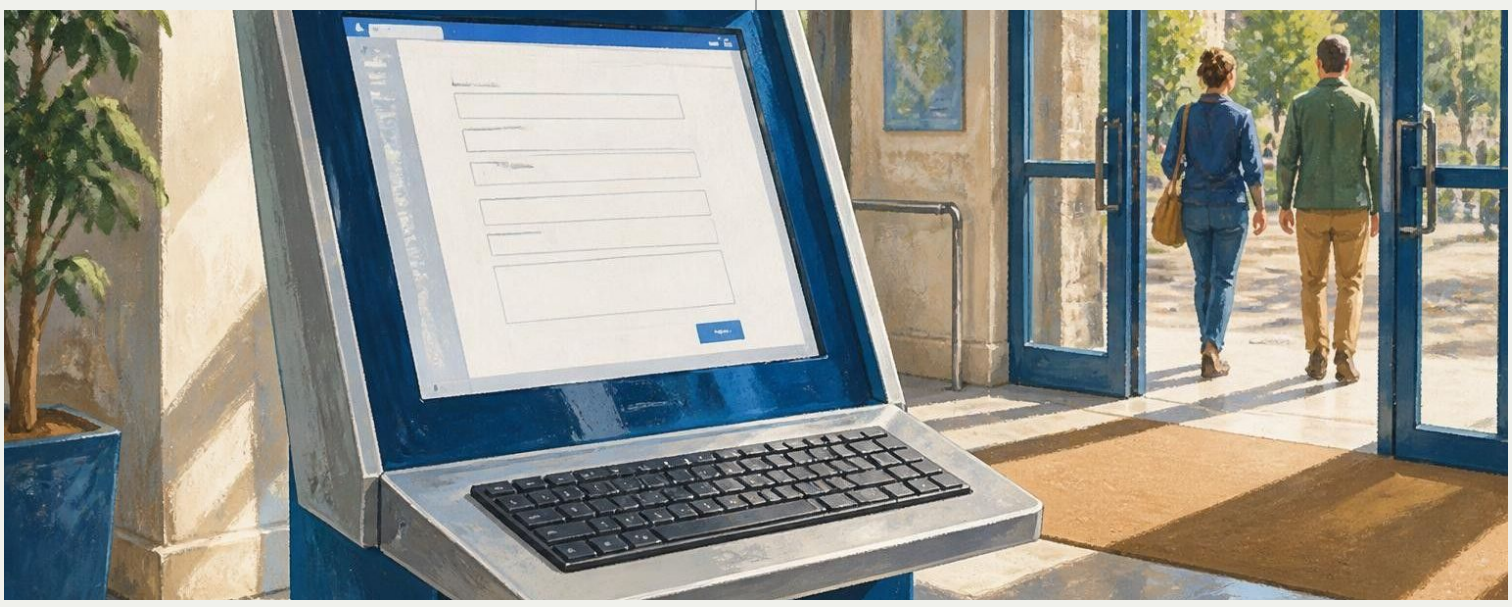
Public exposure, many input channels, and valuable records create a large attack surface.

The request cycle

HTTP carries requests and responses; HTTPS protects them in transit. Cookies often carry session identifiers. APIs expose application functions to browsers and other clients.

Terms to distinguish

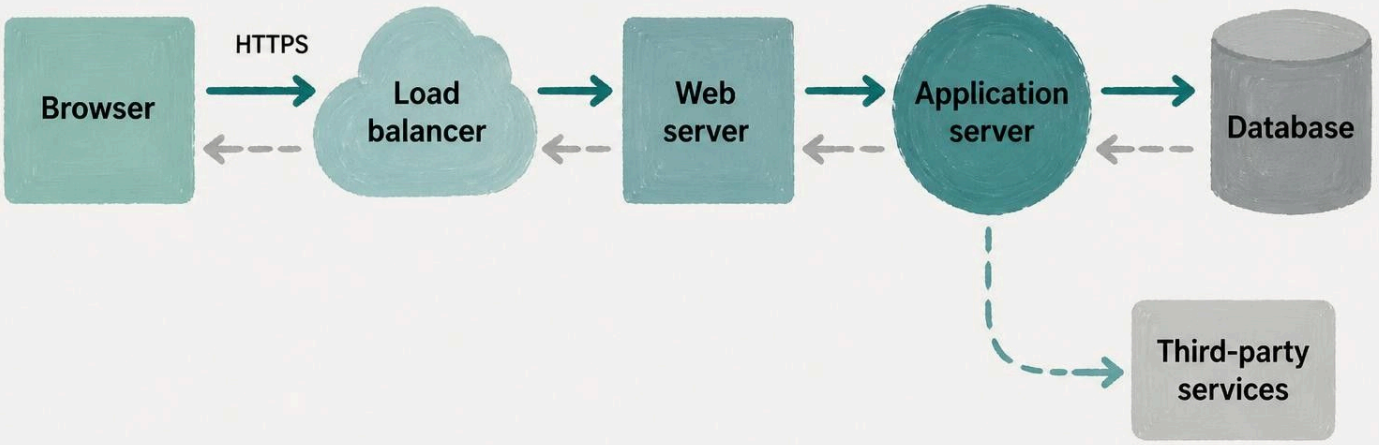
An attack surface is the set of reachable entry points. Input validation checks expected format and meaning. Authentication establishes identity; authorization decides permitted actions.



Browser input is controllable by the user. Client-side checks never establish server-side trust.

Web server architecture shows where weaknesses enter

Trace a request from the browser to stored data. Security depends on every component that receives, forwards, or interprets it.



Typical logical architecture. Roles may share a host or be distributed across services.

Component responsibilities

A load balancer distributes traffic. The web server handles HTTP and static content. The application server executes business logic and queries the database. Third-party services add external dependencies and credentials.

Weakness entry points

- Misconfigured servers expose files or unsafe defaults.
- Outdated components retain known vulnerabilities.
- Exposed admin panels widen privileged access.

> For each hop, ask which identity is trusted, which data crosses the boundary, and where authorization is enforced. Encryption alone cannot correct an unsafe application decision.

The OWASP Top 10 sets the threat landscape

This checklist uses the **OWASP Top 10:2021** categories reflected in this guide. Confirm which edition your certification syllabus requires.

01 **Broken access control** — Users act outside their permitted scope.

02 **Cryptographic failures** — Sensitive data lacks suitable protection.

03 **Injection** — Untrusted data becomes executable instructions.

04 **Insecure design** — Required security controls are missing from the design.

05 **Security misconfiguration** — Unsafe settings expose systems or information.

06 **Vulnerable and outdated components** — Dependencies introduce known or unsupported weaknesses.

07 **Authentication failures** — Identity or session controls allow impersonation.

08 **Integrity failures** — Untrusted updates or data undermine integrity.

09 **Logging gaps** — Attacks go undetected or cannot be investigated.

10 **Server-side request forgery** — A server is induced to contact unintended destinations.

> Use as a checklist for what examiners and testers expect you to know.

Injection and client-side attacks exploit untrusted input

Identify where attacker-controlled data is interpreted and whose authority the resulting action uses.

Code and browser execution

SQL, command, and LDAP injection occur when input changes an interpreter's instructions.

Cross-site scripting (XSS) executes attacker-controlled script in a trusted origin. Stored XSS persists; reflected XSS returns through a response; DOM-based XSS follows an unsafe client-side data flow.

CSRF induces an authenticated browser to submit an unwanted request.

State, files, and services

- Session hijacking misuses a valid session.
- Parameter tampering changes values the server should verify.
- Directory traversal escapes an intended file location.
- File inclusion loads unintended content or code.
- Denial of service exhausts resources; API attacks also target access control and business logic.



Exam distinction: XSS runs script in an origin; CSRF exploits ambient browser credentials.

Hackers follow a repeatable methodology

Authorized testing turns observations into controlled checks, reproducible evidence, and actionable findings.

01 Footprint the infrastructure

Identify servers, technologies, and hidden content within the agreed scope.

02 Analyze the application

Map entry points, parameters, cookies, APIs, and important logic flows.

03 Test identity and access

Check authentication, authorization, and session management across roles and objects.

04 Test input and logic

Evaluate injection points, validation, and business rules using controlled cases.

05 Assess services and report

Test server, API, and web-service exposure. Document impact, evidence, remediation, and retest results.



Scope governs the work, including infrastructure discovery.

> Minimize data access and service disruption. Stop when evidence is sufficient or an agreed safety threshold is reached.

Web app hacking tools automate discovery and exploitation

Choose tools by task and interpret their output. A tool name alone does not establish a vulnerability.

Intercepting proxies

Burp Suite, OWASP ZAP

Inspect and modify HTTP traffic, map application behavior, and support manual testing. Configure scope before active checks.

Injection and content discovery

sqlmap; Gobuster, Nikto, Wfuzz

sqlmap automates SQL injection assessment. Gobuster discovers paths; Nikto checks server issues; Wfuzz varies request inputs.

Credential assessment

Hydra, John the Ripper

Hydra tests online logins. John the Ripper audits password hashes offline. Both require explicit permission and careful handling of credentials.

Technology and service reconnaissance

WhatWeb, Wappalyzer, Nmap

Identify likely web technologies and reachable services. Fingerprints are clues to validate, not proof of exploitable weaknesses.

> Use only on systems you are authorized to test. Set request limits and testing windows; discovery, fuzzing, and login attempts can trigger lockouts or disrupt services.

Countermeasures start with secure input and output handling

Keep data separate from instructions, and enforce security decisions on the server.

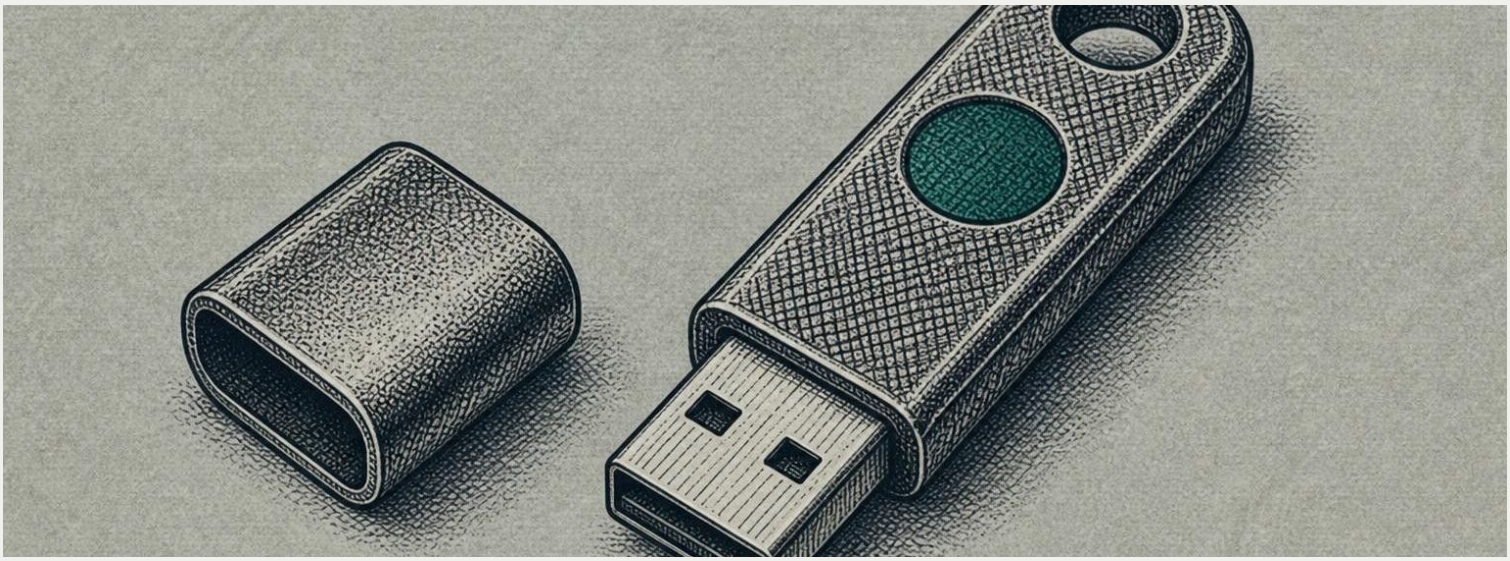
Protect interpretation

- Validate type, length, range, and business meaning using allowlists.
- Sanitize rich content with a trusted library when markup is permitted.
- Use parameterized queries rather than concatenating SQL.
- Encode output for its HTML, attribute, URL, or JavaScript context.

Protect browser state

- Use Content Security Policy as an additional XSS safeguard.
- Apply anti-CSRF tokens and appropriate SameSite cookies.
- Generate unpredictable session IDs, enforce timeouts, and regenerate IDs after login.
- Set Secure and HttpOnly cookie attributes where appropriate.

> Validation, encoding, and parameterization solve different problems. CSP does not replace safe output handling; SameSite does not eliminate every CSRF scenario.



A strong login still needs secure session handling after authentication succeeds.

Defense in depth hardens the whole stack

Independent controls reduce the chance that one weakness becomes a complete compromise.

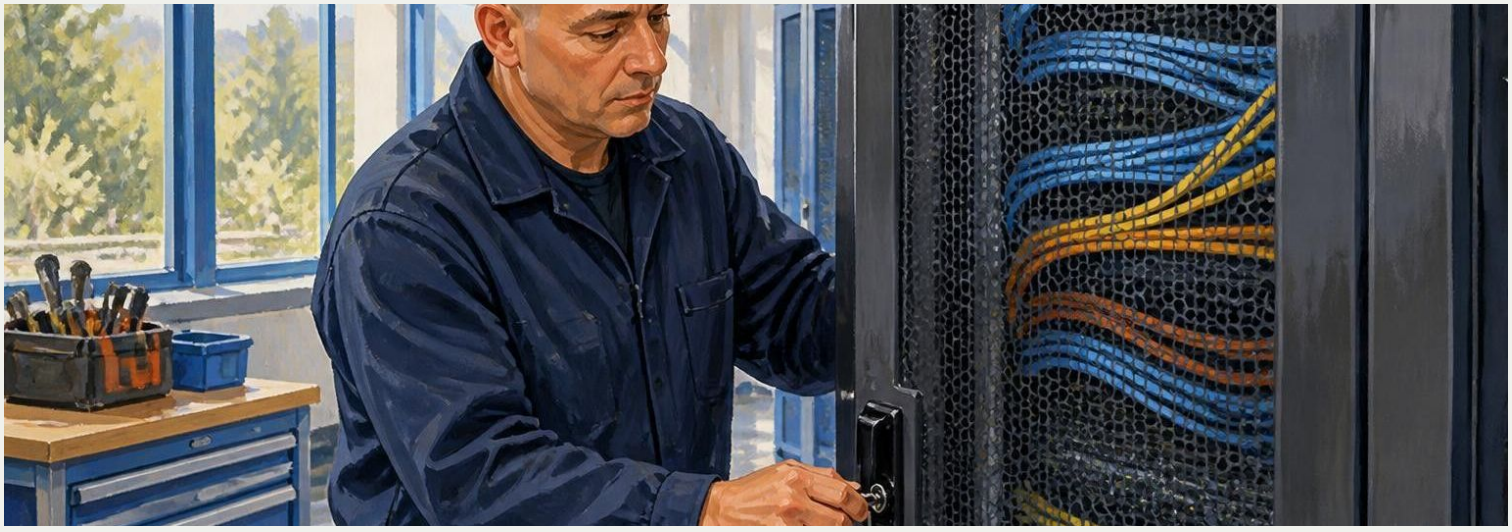
Identity and platform

- Require strong authentication and multi-factor authentication for sensitive access.
- Enforce least privilege and server-side authorization on every protected operation.
- Patch components, harden configurations, and remove default accounts.

Restrict administrative interfaces and service credentials to the access they actually need.

Traffic and operations

- Use TLS everywhere and apply rate limiting.
- Deploy a web application firewall as a compensating layer.
- Log security events, monitor anomalies, and prepare incident response.
- Integrate threat modeling and code review into the secure development lifecycle.



A WAF may filter malicious traffic, but application flaws still require root-cause fixes.

> For exam questions, separate preventive controls from detection and recovery. Useful logs record actor, action, target, and outcome without exposing secrets.

Security testing tools verify defenses before attackers do

Combine runtime checks, code analysis, dependency review, and manual testing. Each reveals a different class of weakness.

DAST: test the running application

OWASP ZAP, Burp Suite Scanner, Acunetix, Netsparker

Dynamic application security testing evaluates reachable behavior through requests and responses. Authenticated coverage depends on configured sessions and reachable workflows. Netsparker is now known as Invicti.

SAST: inspect implementation

SonarQube, Checkmarx, Semgrep

Static application security testing examines source code without exercising the running application. Triage findings for actual data flow, context, and exploitability.

Dependency scanning: inspect components

OWASP Dependency-Check, Snyk

Identify dependencies associated with known vulnerabilities. Confirm versions, applicability, and available remediation.

Manual testing: verify meaning

Test role boundaries, business logic, and multi-step abuse cases that scanners may miss. Record reproducible evidence and retest after fixes.

> A clean scan is not proof of security. Verify the original finding is fixed, related paths remain protected, and regression tests preserve the intended control.